Hybrid Database Framework: Dashboard Enhancement, Performance Evaluation & Final System Packaging

Final Technical Report for CS 432: Databases – Assignment 4

Shardul Junagade Kaushal Bule Akash Gupta

April 18, 2026

Repository: `Kaushal845/Database_1`

Demo Video: `Link`

Contents

1 Executive Summary **2**

2 Project Objective and Scope **2**
2.1 What Was Completed 2
2.2 Deliverables 2

3 System Overview **2**

4 Dashboard Enhancement (Assignment Section 2) **3**

5 Performance Evaluation (Assignment Section 3) **5**
5.1 Experimental Design 5
5.2 Environment and Dataset 6
5.3 Performance Results 6
5.4 Interpretation of Performance Results 8

6 Comparative Analysis (Assignment Section 4) **8**
6.1 Comparative Experiment Design 8
6.2 Discussion: Overhead vs Value 9

7 Final System Packaging (Assignment Section 5) **10**
7.1 Reproducible Setup Paths 10
7.2 Packaging Components Delivered 10
7.3 Docker Packaging Summary 11

8 Limitations **12**

9 Conclusion **12**

1 Executive Summary

This report documents the Assignment 4 completion of our hybrid SQL–MongoDB framework. Building on Assignments 1–3, our goal in this final phase was to deliver a complete and usable logical data system with three outcomes:

- a dashboard that supports operational visibility without exposing backend implementation details,
- a benchmark suite that measures ingestion, query, metadata, and transaction behavior,
- a comparative study that quantifies the cost and value of abstraction versus direct database access.

Our final implementation includes an enhanced React dashboard, benchmark automation scripts, generated technical reports, and reproducible packaging through both local setup and Docker-based deployment.

2 Project Objective and Scope

Assignment 4 required four phases: dashboard enhancement, performance benchmarking, comparative evaluation, and deployable packaging. The scope we implemented is summarized below.

2.1 What Was Completed

- Dashboard enhancement with dedicated tabs for logical entity exploration, query execution, query history, and performance analytics.
- Experimental benchmarking with persisted machine-readable outputs and formatted markdown reports.
- Comparative framework-versus-direct benchmarks across retrieval, nested document access, updates, and insertion.
- System packaging with Docker Compose and documented local setup workflows.

2.2 Deliverables

- Source code repository with Assignment 4 features integrated.
- Performance and comparative benchmark scripts plus generated artifacts.
- Final dashboard with performance visualizations.
- Deployment and execution documentation.

3 System Overview

The framework exposes a logical data layer on top of two physical backends (SQLite and MongoDB). The system is metadata-driven and operation-centric:

1. Ingestion pipeline classifies and routes fields.
2. Metadata store persists placement and schema evolution decisions.

3. Query engine accepts logical CRUD requests and executes backend-specific plans.
4. Transaction coordinator enforces atomic cross-backend behavior via two-phase commit when enabled.
5. Dashboard API serves logical summaries, entities, records, session activity, query history, and benchmark outputs.

This architecture preserves abstraction for application developers while still allowing measurable, explainable execution across heterogeneous storage.

4 Dashboard Enhancement (Assignment Section 2)

Implemented Dashboard Capabilities:
The dashboard was enhanced to satisfy all requested capabilities from Assignment 4.

Requirement	Implementation Status and Evidence
Viewing active sessions	Implemented in Session view. Shows session start time, last update, duration, and activity statistics.
Listing logical entities within a session	Implemented in Entity Catalog. Lists discovered logical entities (nested and repeating), with filters and summaries.
Viewing instances of each entity	Implemented in Entity Catalog via instance counts per entity and aggregate counters.
Inspecting field names and values of logical objects	Implemented across Records and Entity Catalog. Records view supports per-record field expansion; Entity Catalog includes field-level metadata and samples.
Displaying results of executed logical queries	Implemented in Query Builder with full CRUD execution and rendered JSON result payloads.
Viewing query execution history	Implemented in Query History with timestamp, operation, filters, latency, status, and record counts, plus summary metrics and clear-history action.

Constraint Compliance: Logical View Without Backend Leakage
The assignment constraint requires that users interact with logical abstractions rather than physical storage details. We reviewed and adjusted UI wording so regular dashboard views avoid exposing implementation-level terms like SQL table names, MongoDB collection names, index internals, or placement internals.
Technical comparative views (Performance tab) intentionally mention direct SQL and MongoDB because Assignment 4 Sections 3 and 4 explicitly require that comparison.

Dashboard Screenshots:

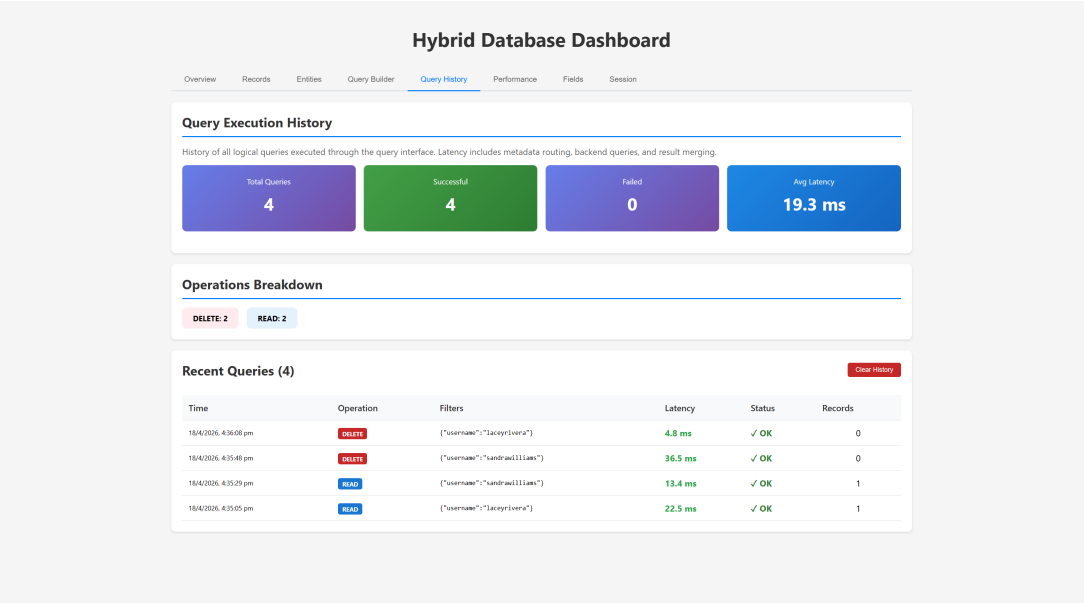

Figure 1: Query execution history in the enhanced dashboard.

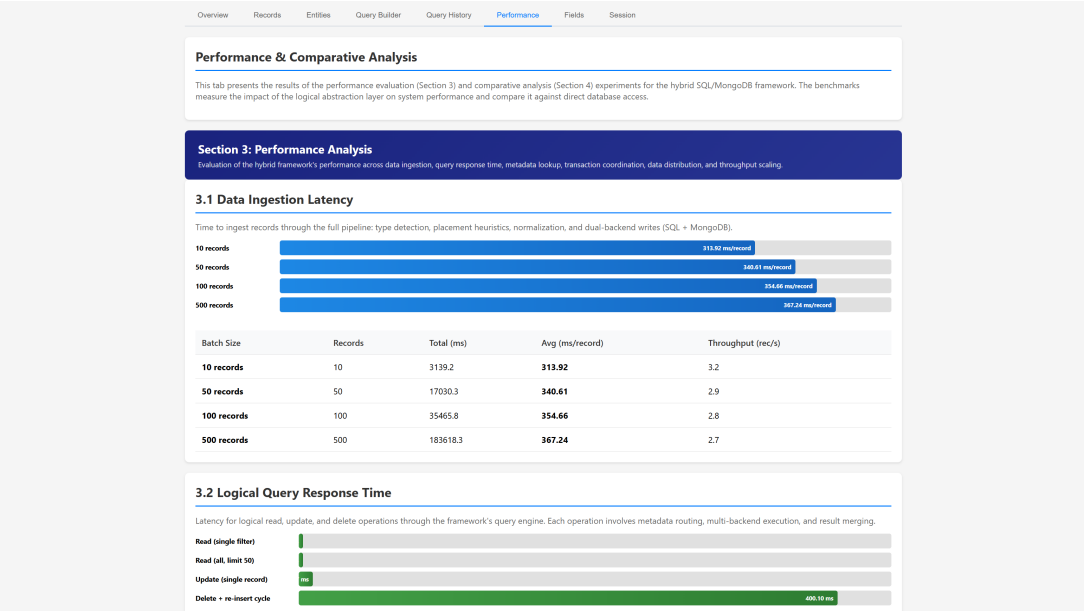

Figure 2: Performance analytics integrated into the dashboard.

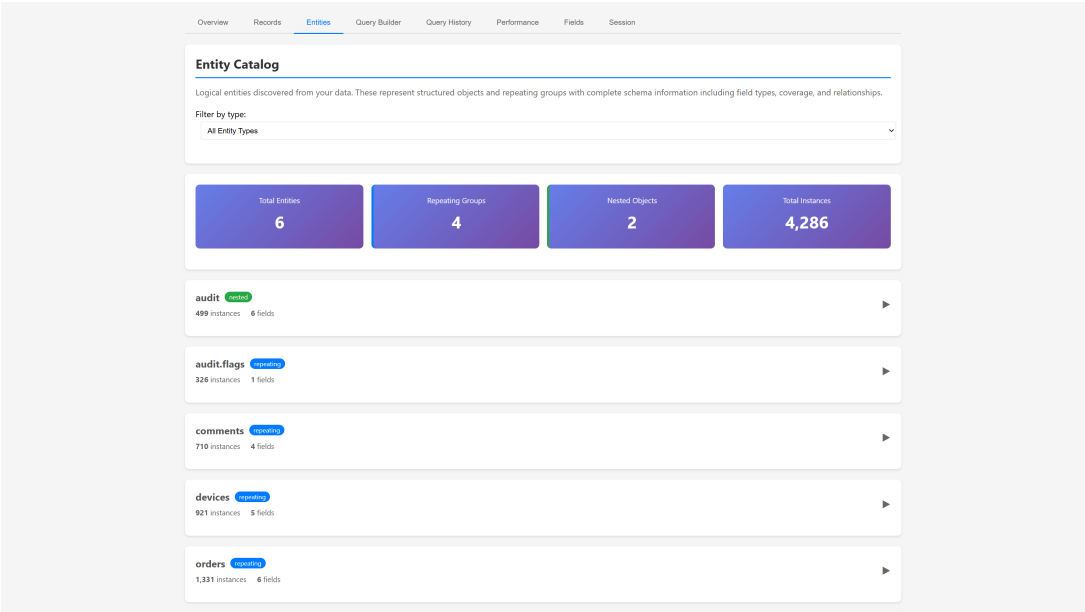

Figure 3: Entity catalog view in the enhanced dashboard.

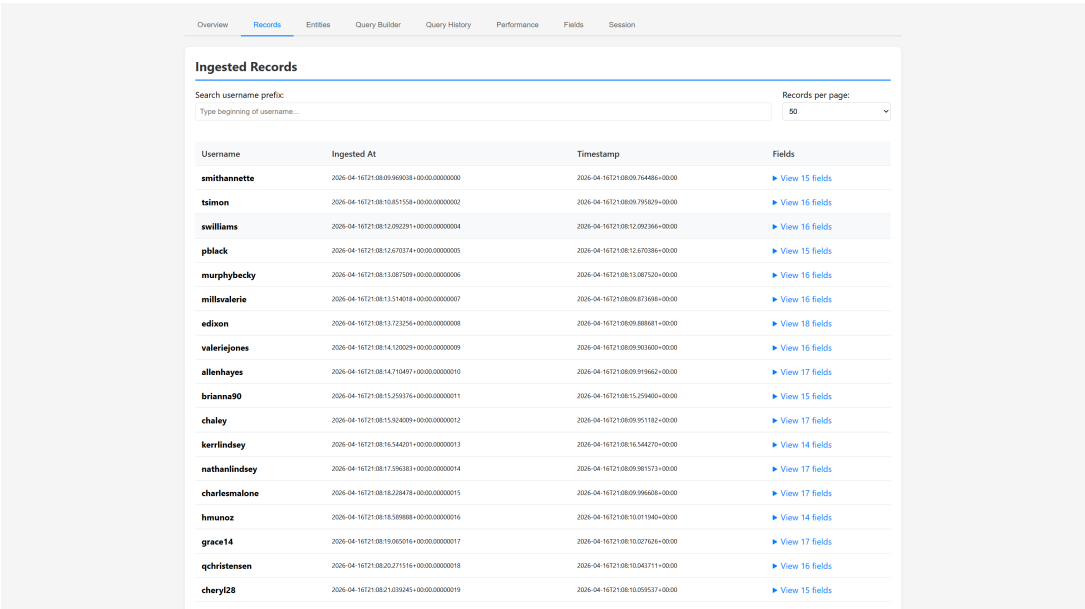

Figure 4: Record inspector view in the enhanced dashboard.

5 Performance Evaluation (Assignment Section 3)

5.1 Experimental Design

To evaluate the hybrid framework, we implemented a repeatable benchmark suite focused on assignment-mandated dimensions:

- data ingestion latency,
- logical query response time,

- metadata lookup overhead,
- transaction coordination overhead across SQL and MongoDB.

In addition, we collected:

- average query latency and tail metrics,
- throughput in operations per second,
- distribution of fields and records across backends.

5.2 Environment and Dataset

- Data generator: Faker-based synthetic records with scalar fields and nested structures.
- Backends: SQLite (embedded) and MongoDB.
- Runtime: Python 3.13.7.
- Seed size for query benchmarks: 100 records.

Note: Replace with machine details before final PDF export.

- CPU: **Intel i7-13700H**
- RAM: **16 GB**
- OS: **Windows 11**

5.3 Performance Results

Execution of benchmark suites generating:

- `docs/performance_benchmark_results.json`,
- `docs/comparative_benchmark_results.json`,
- `docs/performance_report.md`,
- `docs/comparative_report.md`.

5.3.1 Ingestion Latency and Throughput

Batch Size	Total (ms)	Avg (ms/record)	Throughput (rec/s)
10	3139.2	313.924	3
50	17030.3	340.605	3
100	35465.8	354.658	3
500	183618.3	367.237	3

Table 2: Ingestion benchmark results.

5.3.2 Logical Query Response Time

Operation	Avg (ms)	Min (ms)	Max (ms)	P95 (ms)	Iters
Read (single filter)	1.72	1.16	5.35	2.87	50
Read (all, limit 50)	3.05	1.85	5.53	4.55	30
Update (single record)	10.47	2.87	21.68	17.79	30
Delete + re-insert cycle	400.10	255.11	816.63	702.26	20

Table 3: Logical query latency across operations.

5.3.3 Metadata Lookup Overhead

Operation	Avg (ms)	Iterations
Field mapping lookup	0.0001	200
Placement decision lookup	0.0001	200

Table 4: Metadata lookup overhead.

5.3.4 Transaction Coordination Overhead

Operation	Avg (ms)	Iterations
Full 2PC (begin → prepare → commit)	2.44	20
Direct update (no 2PC)	2.75	20

Table 5: 2PC coordination overhead versus direct update.

5.3.5 Data Distribution Across Backends

Backend	Fields Stored
SQL	3
MongoDB	7
Both	1
Buffer	0
Total	**8**

Table 6: Field distribution across storage backends.

Storage	Record Count
SQL (main table)	100
MongoDB (main collection)	100
Buffer (pending)	0

Table 7: Record distribution across storage objects.

5.3.6 Throughput Under Increasing Workload

Workload (ops)	Elapsed (s)	Throughput (ops/s)
10	0.027	372.1
25	0.089	281.6
50	0.118	423.7
100	0.372	268.7

Table 8: Read throughput scaling.

5.4 Interpretation of Performance Results

The measurements are consistent with a metadata-driven, dual-write architecture:

- Ingestion cost remains the dominant write overhead because one logical write can trigger classification, metadata updates, normalization handling, and writes to two backends.
- Query latency stays low in absolute terms for this dataset size; however, update and delete+reinsert costs are higher due to coordination and consistency workflows.
- Metadata lookups are effectively negligible relative to query and ingest costs.
- Throughput trends are broadly stable under moderate read load, indicating practical scalability for assignment-scale workloads.

6 Comparative Analysis (Assignment Section 4)

6.1 Comparative Experiment Design

We compared logical framework operations against direct backend access to quantify abstraction cost and benefit. The implemented comparisons were:

1. Record retrieval: framework read vs direct SQL SELECT vs direct MongoDB find.

Method	Avg (ms)	Min (ms)	Max (ms)	P95 (ms)
Framework (logical query)	2.403	1.885	3.899	3.088
Direct SQL (SELECT)	0.023	0.020	0.080	0.025
Direct MongoDB (find)	1.161	0.705	1.800	1.561

Table 9: Record retrieval latency breakdown.

2. Nested document access: framework read vs direct MongoDB find with projection.

Method	Avg (ms)	Min (ms)	Max (ms)	P95 (ms)
Framework (logical query)	3.362	2.233	5.619	4.853
Direct MongoDB (find + projection)	1.487	1.131	3.045	2.028

Table 10: Nested document access latency breakdown.

3. Record update: framework logical update vs direct SQL UPDATE vs direct MongoDB $set.

Method	Avg (ms)	Min (ms)	Max (ms)	P95 (ms)
Framework (logical update)	8.815	6.511	16.013	11.075
Direct SQL (UPDATE)	0.022	0.014	0.209	0.017
Direct MongoDB ($set)	1.204	0.914	1.627	1.582

Table 11: Record update latency breakdown.

4. Record insertion: framework ingest vs direct SQL INSERT vs direct MongoDB insert_one.

Method	Avg (ms)	Min (ms)	Max (ms)	P95 (ms)
Framework (full pipeline)	366.768	274.711	1237.725	372.490
Direct SQL (INSERT)	0.103	0.067	0.270	0.182
Direct MongoDB (insert_one)	1.032	0.560	1.645	1.567

Table 12: Record insertion latency breakdown.

Comparative Results Summary:

Test	Framework (ms)	Direct SQL (ms)	Direct Mon
Record Retrieval (single filter read)	2.40	0.02	
Nested Document Access (orders + profile)	3.36	N/A	
Record Update (single field)	8.81	0.02	
Record Insertion	366.77	0.10	

Table 13: Comparative summary (framework vs direct access).

6.2 Discussion: Overhead vs Value

Where the abstraction adds overhead

- **Read operations**: The framework performs metadata lookups, routes queries to both SQL and MongoDB, and merges results. This adds measurable latency compared to a single direct query.

- **Ingestion**: The framework runs type detection, placement heuristics, normalization, and dual-backend writes, which is significantly slower than a single raw INSERT.
- **Updates**: The framework routes fields to the correct backend based on metadata, potentially updating SQL and MongoDB separately. Direct updates on a single backend are naturally faster.

Where the abstraction provides value

- **Unified access**: Users query a single logical interface without knowing which backend stores each field.
- **Automatic schema evolution**: New fields are automatically detected, classified, and routed to an appropriate backend.
- **Nested entity management**: Arrays can be normalized into child tables while embedded documents remain document-native in MongoDB, all transparently.
- **ACID guarantees**: The two-phase commit model provides atomic behavior across both backends.

When to use framework vs direct access

Scenario	Recommended Approach
Schema-flexible, evolving data	Framework
High-throughput, fixed-schema writes	Direct backend
Cross-backend queries (SQL + MongoDB)	Framework
Single-backend, latency-critical reads	Direct backend
Need ACID across SQL + MongoDB	Framework

Table 14: Decision guide: abstraction layer vs direct backend access.

7 Final System Packaging (Assignment Section 5)

7.1 Reproducible Setup Paths

The framework is packaged with two setup strategies to support different evaluator environments:

- **Docker-based setup** for one-command reproducibility,
- **Local setup** for development and debugging.

7.2 Packaging Components Delivered

Required Item	What is Provided
Source code repository	GitHub repository with backend, frontend, benchmark scripts, tests, and docs.

Setup instructions for dependencies	Python and Node dependency docs, runnable command references.
SQL and MongoDB configuration guidance	SQLite defaults documented; MongoDB local and Docker startup documented.
Run ingestion API instructions	API startup and ingestion flow documented.
Run logical query interface instructions	Query Builder and API endpoint workflows documented.
Launch dashboard instructions	Local frontend workflow and Docker integrated dashboard workflow documented.

7.3 Docker Packaging Summary

Detailed Docker instructions are provided in `DOCKER_SETUP.md`.
Using `docker compose up -build` starts:

- MongoDB container,
- App container serving FastAPI and built dashboard.

Exposed endpoints include dashboard UI, API docs, and health endpoint. The Docker guide also includes ingestion commands, query examples, benchmark execution commands, and TA demonstration flow.

Deployment and API Evidence Screenshots:

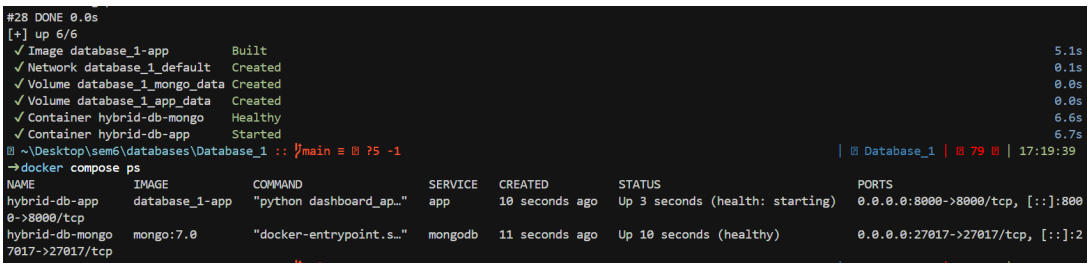

Figure 5: Dockerized deployment.

Figure 6: API evidence from benchmark endpoint output.

8 Limitations

- Some percentage-overhead values can look unintuitive when direct baseline latency is extremely small.
- Current deployment is single-node and primarily optimized for coursework-scale data volumes.
- Query history is currently maintained in memory at the API layer, so it is cleared when the service restarts.

9 Conclusion

Assignment 4 transformed the project from a prototype data pipeline into a complete logical data platform with measurable behavior and reproducible execution. The dashboard now supports end-user observability and interaction, benchmarks quantify system costs and behavior, comparative analysis explains the abstraction trade-offs, and packaging enables straightforward deployment.

In short, the final system demonstrates the practical value of a metadata-driven logical layer over heterogeneous stores: measurable overhead exists, but so does meaningful reduction in application complexity, improved evolvability, and stronger consistency guarantees across backends.